

MODULE 04/09

PostgreSQL & SQL

La base de données relationnelle de référence

■ Durée : 1 semaine

■ Projet : Social Seller

■ Niveau : Débutant →
Opérationnel

Objectifs de la semaine

- ✓ Comprendre le modèle relationnel : tables, colonnes, clés primaires et étrangères
- ✓ Écrire des requêtes SELECT, INSERT, UPDATE, DELETE
- ✓ Maîtriser les JOIN pour combiner des tables
- ✓ Créer des index, des triggers et des fonctions plpgsql
- ✓ Lire et écrire les migrations SQL de Social Seller

Planning de la semaine

JOUR 1	Modèle relationnel	2-3h
	• Tables, colonnes, types de données : text, integer, boolean, uuid, timestampz • Clé primaire (PRIMARY KEY) et clé étrangère (REFERENCES) • Contraintes : NOT NULL, UNIQUE, CHECK • CREATE TABLE et ALTER TABLE	
JOUR 2	Requêtes de base	2-3h
	• SELECT * FROM / SELECT colonnes FROM • WHERE, AND, OR, IN, IS NULL • ORDER BY, LIMIT, OFFSET — pagination • INSERT INTO, UPDATE SET WHERE, DELETE WHERE	
JOUR 3	JOINS et agrégations	2-3h
	• INNER JOIN, LEFT JOIN — quelle différence ? • COUNT(*), SUM(), AVG(), MAX(), MIN() • GROUP BY et HAVING • Sous-requêtes : SELECT ... WHERE id IN (SELECT ...)	
JOUR 4	Index, triggers et fonctions	2-3h
	• CREATE INDEX — pourquoi et quand indexer • Transactions : BEGIN, COMMIT, ROLLBACK • Fonctions plpgsql : CREATE OR REPLACE FUNCTION • Triggers : AFTER INSERT/UPDATE — automatiser une action	
JOUR 5	Mini-projet de la semaine	3-4h
	• Concevoir et créer un schéma 'boutique en ligne' : products, orders, order_items • Insérer 10 produits et 5 commandes avec leurs items	

- Requête : top 3 produits les plus vendus (JOIN + GROUP BY + ORDER BY)
- Créer un trigger qui met à jour stock_quantity après une commande
- Créer un index sur orders.status et mesurer la différence

Concepts clés détaillés

Créer une table (comme dans Social Seller)

```
-- Table conversations (extrait de 001_initial_schema.sql)
CREATE TABLE public.conversations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES public.tenants(id),
  platform text NOT NULL CHECK (platform IN ('whatsapp', 'facebook', 'tiktok')),
  customer_name text,
  customer_id text,
  status text NOT NULL DEFAULT 'new'
  CHECK (status IN ('new', 'in_progress', 'resolved')),
  created_at timestamptz NOT NULL DEFAULT now()
);

-- Index pour accélérer les requêtes par tenant
CREATE INDEX conversations_tenant_id_idx ON public.conversations(tenant_id);
```

Les requêtes du quotidien

```
-- Récupérer les conversations d'un tenant, triées par date
SELECT id, platform, customer_name, status, created_at
FROM public.conversations
WHERE tenant_id = '550e8400-e29b-41d4-a716-446655440000'
AND status != 'resolved'
ORDER BY created_at DESC
LIMIT 20 OFFSET 0;

-- Compter les messages non lus par conversation
SELECT conversation_id, COUNT(*) AS unread_count
FROM public.messages
WHERE direction = 'inbound'
AND is_read = false
AND tenant_id = '550e8400-...'
```

```

GROUP BY conversation_id;

-- Mise à jour : marquer comme lu
UPDATE public.messages
SET is_read = true
WHERE conversation_id = 'abc-123'
AND direction = 'inbound'
AND is_read = false;

```

JOIN — combiner plusieurs tables

```

-- Top produits vendus (de stats.ts dans Social Seller)
SELECT
p.id AS product_id,
p.name,
SUM(oi.quantity) AS total_sold
FROM public.order_items oi
INNER JOIN public.orders o ON o.id = oi.order_id
INNER JOIN public.products p ON p.id = oi.product_id
WHERE o.tenant_id = $1
AND o.status = 'delivered'
GROUP BY p.id, p.name
ORDER BY total_sold DESC
LIMIT 5;

-- LEFT JOIN : garder les conversations MÊME sans messages
SELECT c.id, c.customer_name, m.content AS last_message
FROM public.conversations c
LEFT JOIN public.messages m ON m.id = (
SELECT id FROM public.messages
WHERE conversation_id = c.id
ORDER BY created_at DESC LIMIT 1
)
WHERE c.tenant_id = $1
ORDER BY c.updated_at DESC;

```

Fonction plpgsql — transaction atomique

```

-- Extrait de 012_order_status_transitions.sql
CREATE OR REPLACE FUNCTION public.transition_order_status(

```

```

p_order_id uuid,
p_tenant_id uuid,
p_new_status text,
p_user_id uuid,
p_cancelled_reason text DEFAULT NULL
)
RETURNS public.orders
LANGUAGE plpgsql
AS $$
DECLARE
v_order public.orders;
BEGIN
-- Verrouiller la ligne pour éviter les race conditions
SELECT * INTO v_order FROM public.orders
WHERE id = p_order_id FOR UPDATE;

IF v_order.id IS NULL THEN
RAISE EXCEPTION 'order_not_found';
END IF;

-- Mettre à jour le statut
UPDATE public.orders SET status = p_new_status WHERE id = p_order_id
RETURNING * INTO v_order;

RETURN v_order;
END;
$$;

```

■ Toutes les opérations dans une fonction plpgsql s'exécutent dans la MÊME transaction. Si une ligne échoue, tout est annulé automatiquement. C'est pour ça que Social Seller utilise des RPCs pour les opérations critiques (stock, statut commande).

Exercices pratiques

Exercice 1 — Schéma boutique (30 min)

- Créer les tables : products (id, name, price, stock), orders (id, status, created_at), order_items (order_id, product_id, quantity, unit_price)
- Ajouter les contraintes : price >= 0, quantity > 0, status IN ('new','confirmed','delivered')
- Créer les index sur orders.status et order_items.order_id

Exercice 2 — Requêtes analytiques (45 min)

- Requête 1 : total des ventes par produit (SUM + GROUP BY)
- Requête 2 : commandes du mois courant avec leur montant total (JOIN + WHERE + DATE_TRUNC)
- Requête 3 : produits jamais commandés (LEFT JOIN + IS NULL)

Exercice 3 — Trigger de stock (45 min)

- Créer une fonction qui décrémente stock quand une commande passe en 'delivered'
 - Créer le trigger AFTER UPDATE ON orders
 - Tester : mettre une commande en delivered, vérifier que le stock a baissé
-

Ressources pour aller plus loin

- Documentation PostgreSQL : postgresql.org/docs/current
- SQL interactif : pgexercises.com
- Explain Analyze (comprendre les query plans) : explain.dalibo.com
- Supabase SQL Editor : pour pratiquer directement sur ton projet